Pines: función, resistencias y drenador abierto

Fuente: UM10360 rev. 4.1, cap. 8, tablas 77-99, pp. 115-129; cap. 4, tabla 46.

Acceso directo con **LPC17xx.h**. Se retiraron las funciones y estructuras de drivers; se conservan las funciones NVIC. R/W = lectura/escritura; RO = solo lectura; WO = solo escritura; W1C = escribir 1 limpia. Los valores de reset corresponden a los campos implementados. **Los bits reservados no deben escribirse en 1.**

LPC_PINCON->PINSELn / PINMODEn

Para Px.y: n = 2·x + (y // 16); desplazamiento s = 2·(y % 16). Cada campo implementado ocupa [s+1:s]. La fórmula localiza el campo; no implica que existan todos los pines o registros.

Registro	Campos / reset	Significado
PINSELn	2 bits/pin; R/W; 00	00 = GPIO en los pines listados. 01, 10 y 11 eligen la función indicada en la tabla de ese pin; algunas combinaciones están reservadas.
PINMODEn	2 bits/pin; R/W; 00	00 = pull-up; 01 = repeater; 10 = sin pull-up/pull-down; 11 = pull-down.
PINMODE_ODx	1 bit/pin; R/W; 0	0 = salida normal; 1 = open-drain. Con dato 0 conduce a bajo; con dato 1 libera la salida. Solo campos implementados.
PINSEL10	bit 3; R/W; 0	Excepción a los campos de 2 bits: 0 = trace deshabilitado; 1 = trace habilitado en P2.2-P2.6.

Repeater: mantiene débilmente el último nivel con pull-up o pull-down. Reduce el riesgo de entrada flotante; no garantiza consumo nulo. En open-drain, PINMODE actúa cuando la salida está liberada.

Puerto	Pines implementados (LPC1769, 100 pines)	Registros de selección/modo usados
P0	0-11 y 15-30	PINSEL0/1; PINMODE0/1
P1	0-1, 4, 8-10 y 14-31	PINSEL2/3; PINMODE2/3
P2	0-13	PINSEL4; PINMODE4
P3	25-26	PINSEL7; PINMODE7
P4	28-29	PINSEL9; PINMODE9

Excepciones que afectan la configuración

P0.27 y P0.28: son open-drain dedicados de I²C, sin pull-up/pull-down configurable; PINMODE1[25:22] y PINMODE_OD0[28:27] no los configuran. **P0.29 y P0.30:** tampoco tienen resistencias seleccionables; PINMODE1[29:26] está reservado. Ambos deben tener la misma dirección GPIO.

Campos de pines no implementados: reservados. En PINMODE_OD0 también están reservados 14:12, 28:27 y 31. En los otros puertos solo se usan los bits de los pines de la tabla. La disponibilidad física en una placa puede ser menor que la del encapsulado.

Al modificar campos de varios bits, borrar primero la máscara y luego escribir el valor. Un OR por sí solo no reemplaza una función previamente seleccionada. PCONP[15] (PCGPIO, reset 1) alimenta el bloque de pines, GPIO e interrupciones GPIO.

GPIO e interrupciones de puertos

Fuente: cap. 9, tablas 102-124, pp. 132-149; cap. 5, tabla 50; LPC17xx.h.

LPC_GPIOx: x = 0..4. Cada bit y corresponde a Px.y; solo tienen uso los pines implementados de la página anterior. Los registros son de 32 bits, pero eso no crea 32 pines por puerto.

Registro / acceso	Bits y reset	Operación
FIODIR / R/W	31:0; 0	0 = entrada; 1 = salida. Solo controla la dirección cuando PINSEL elige GPIO.
FIOMASK / R/W	31:0; 0	0 = permite acceso al pin. 1 = bloquea escrituras a FIOPIN, FIOSET y FIOCLR; la lectura de ese bit en FIOPIN da 0. No es una máscara de interrupciones.
FIOPIN / R/W	31:0; tabla: 0	Lectura: nivel físico, sujeto a FIOMASK; depende del circuito externo. Escritura: coloca 0/1 en las salidas no enmascaradas. No asumir que todas las entradas físicas están en 0 después de reset.
FIOSET / R/W	31:0; 0	Escribir 1 pone la salida en alto; escribir 0 no cambia nada. Respeta FIOMASK. Al leer devuelve el latch de salida, no el nivel físico impuesto externamente.
FIOCLR / WO	31:0; 0	Escribir 1 pone la salida en bajo; escribir 0 no cambia nada. Respeta FIOMASK. Usar asignación directa, no lectura-modificación-escritura.

P0.29/P0.30: si cualquiera de FIODIR[29:30] es 0, ambos pines son entradas; para que ambos sean salidas, los dos bits deben ser 1. FIOPIN puede leer el nivel digital aun con otra función seleccionada, salvo entradas configuradas como ADC.

LPC_GPIOINT: solo puertos 0 y 2

En IOx..., x solo puede ser 0 o 2. P0: bits 0-11 y 15-30; reservados 14:12 y 31. P2: bits 13:0; reservados 31:14. Todos los campos de interrupción arrancan en 0.

Registro / acceso	Bits	Significado
IOxIntEnR / R/W	pines válidos	1 habilita interrupción por flanco ascendente; 0 deshabilita.
IOxIntEnF / R/W	pines válidos	1 habilita interrupción por flanco descendente; 0 deshabilita. Puede coexistir con EnR.
IOxIntStatR / RO	pines válidos	1 indica interrupción por flanco ascendente en el pin.
IOxIntStatF / RO	pines válidos	1 indica interrupción por flanco descendente en el pin.
IOxIntClr / WO	pines válidos	Escribir 1 limpia simultáneamente StatR y StatF del pin. Escribir 0 no actúa.
IntStatus / RO	0 = P0Int 2 = P2Int	1 indica alguna interrupción pendiente en ese puerto. Bit 1 y bits 31:3 reservados.

```
NVIC_EnableIRQ(EINT3_IRQn);
void EINT3_IRQHandler(void) { /* atender y limpiar las fuentes */ }
```

EINT3 comparte vector con las interrupciones GPIO. Identificar la fuente con IntStatus/IOxIntStatR/F y, si se usa la entrada EINT3, con EXTINT[3]. Limpiar la fuente correspondiente. NVIC_ClearPendingIRQ() no reemplaza esta limpieza.

Interrupciones externas y NVIC

Fuente: cap. 3, §3.6, tablas 9-12, pp. 26-29; cap. 5; tabla 84; CMSIS adjunto.

Entrada	Pin	Campo PINSEL4	Función
EINT0	P2.10	21:20	01
EINT1	P2.11	23:22	01
EINT2	P2.12	25:24	01
EINT3	P2.13	27:26	01

LPC_SC: campos 3:0, un bit por EINT; reset 0

Registro	Campo	Operación
EXTINT / R/W (W1C)	bit i = EINTi	1 indica evento. Escribir 1 lo limpia; escribir 0 no actúa. Por nivel, solo se puede limpiar cuando el pin ya está inactivo.
EXTMODE / R/W	bit i = EXTMODEi	0 = sensibilidad por nivel; 1 = sensibilidad por flanco.
EXTPOLAR / R/W	bit i = EXTPOLARi	0 = nivel bajo o flanco descendente; 1 = nivel alto o flanco ascendente. Depende de EXTMODE.

i = 0, 1, 2 o 3. En los tres registros, 31:4 está reservado. Para cambiar modo/polaridad: deshabilitar la IRQ en NVIC, configurar, limpiar EXTINT escribiendo 1 y volver a habilitar. Si procede, limpiar también el pendiente del NVIC después de atender la fuente.

Funciones CMSIS del NVIC conservadas

Función	Uso
NVIC_EnableIRQ(IRQn)	Habilita la IRQ del periférico en el controlador.
NVIC_DisableIRQ(IRQn)	Deshabilita la IRQ en el controlador.
NVIC_ClearPendingIRQ(IRQn)	Limpia el pendiente en NVIC; no limpia el flag del periférico.
NVIC_SetPendingIRQ(IRQn)	Coloca una IRQ externa en pendiente por software.
NVIC_GetPendingIRQ(IRQn)	Devuelve 1 si está pendiente; 0 si no.
NVIC_GetActive(IRQn)	Devuelve 1 si la IRQ está activa.
NVIC_SetPriority(IRQn, prioridad)	5 bits implementados: valores 0..31; menor número = mayor prioridad. Pasar el valor sin desplazar.
NVIC_GetPriority(IRQn)	Devuelve el valor lógico de prioridad.

Las operaciones Enable/Disable/Pending/Active anteriores se usan con IRQn de periféricos (no negativos). **SysTick se habilita con CTRL.TICKINT**; sí admite NVIC_SetPriority(SysTick_IRQn, p). La división entre preempción y subprioridad depende de PRIGROUP.

Fuente	IRQn de CMSIS	Handler (retorno void, argumento void)
EINT0..3	EINT0_IRQn .. EINT3_IRQn	EINT0_IRQHandler .. EINT3_IRQHandler
Timers 0..3	TIMER0_IRQn .. TIMER3_IRQn	TIMER0_IRQHandler .. TIMER3_IRQHandler
SysTick	SysTick_IRQn	SysTick_Handler
ADC / DMA	ADC_IRQn / DMA_IRQn	ADC_IRQHandler / DMA_IRQHandler

Los handlers son funciones que define el programa para atender interrupciones; no son drivers. Las expresiones con x o i son notación genérica: reemplazarlas por el periférico o bit concreto.

SysTick: contador descendente de 24 bits

Fuente: cap. 23, tablas 439-443, pp. 515-519; core_cm3.h; ARM DUI 0552A, §4.4.

SysTick->CTRL (STCTRL en UM10360)

Bit	Nombre / acceso / reset	Significado
0	ENABLE / R/W / 0	1 = habilita contador; 0 = deshabilita.
1	TICKINT / R/W / 0	1 = solicita excepción SysTick al contar de 1 a 0.
2	CLKSOURCE / R/W / 1	1 = CCLK (CPU); 0 = entrada externa STCLK en P3.26.
16	COUNTFLAG / RO / 0	1 si llegó a 0 desde la última lectura de CTRL. Leer CTRL o escribir VAL limpia esta bandera.
15:3 y 31:17	Reservados	No escribir en 1.

Recarga, valor actual y calibración

Registro / acceso	Campos y reset	Operación
LOAD / R/W	23:0 RELOAD = 0 31:24 reservados	Valor de recarga. Tras llegar a cero, se recarga en el siguiente flanco. Para N ciclos, escribir N-1.
VAL / R/W especial	23:0 CURRENT = 0 31:24 reservados	Lectura: cuenta actual. Escribir cualquier valor pone CURRENT y COUNTFLAG en 0; no carga ese valor en el contador.
CALIB / RO (*)	23:0 TENMS = 0x0F423F 29:24 reservados	999999: recarga para 10 ms a 100 MHz. No se adapta automáticamente a la frecuencia utilizada.
CALIB / RO (*)	30 SKEW = 0	0 = TENMS preciso para la condición indicada; 1 = aproximado.
CALIB / RO (*)	31 NOREF = 0	0 = referencia externa disponible; 1 = sin referencia externa. No indica que haya una señal conectada físicamente.

(*) **Discrepancia documental:** la tabla 439 de UM10360 dice R/W para STCALIB. El CMSIS adjunto lo declara __I (solo lectura) y ARM lo define RO. Para programar con CMSIS, tratar CALIB como solo lectura.

Fuente externa y límites

STCLK es **P3.26, PINSEL7[21:20] = 01**. En este LPC1769, CLKSOURCE=0 selecciona ese pin. El manual limita f_STCLK a CCLK/4. El valor TENMS de 100 MHz se utiliza normalmente con el reloj de CPU; no autoriza a exceder el límite del pin externo.

Período: T = (LOAD + 1) / f_SysTick. Para interrupciones periódicas, LOAD = 1..0xFFFFFF; N = 2..2^{24} ciclos. LOAD=0 no produce la transición periódica 1→0. T máximo = 2^{24} / f_SysTick (167,77216 ms a 100 MHz).

Configuración directa: ejemplo con CCLK conocido

Atender en **void SysTick_Handler(void)**. COUNTFLAG y la excepción pendiente son estados distintos: no hace falta leer COUNTFLAG para reconocer la entrada al handler. Evitar lecturas accidentales de CTRL si se utiliza COUNTFLAG para polling.

No se incluye SysTick_Config ni las funciones SYSTICK_...: se usa la configuración explícita por registros. Los valores de reset mostrados son los publicados por NXP para el LPC176x/5x; inicializar LOAD y VAL antes de arrancar.

Timers 0-3: pines, reloj y conteo

Fuente: tablas 40-42, 46, 80-86; cap. 21, tablas 426-429, pp. 501-506.

Timer	CAP / función	MAT / función
0	CAP0.0: P1.26 / 11 CAP0.1: P1.27 / 11	MAT0.0: P1.28 / 11 o P3.25 / 10 MAT0.1: P1.29 / 11 o P3.26 / 10
1	CAP1.0: P1.18 / 11 CAP1.1: P1.19 / 11	MAT1.0: P1.22 / 11 MAT1.1: P1.25 / 11
2	CAP2.0: P0.4 / 11 CAP2.1: P0.5 / 11	MAT2.0: P0.6 / 11 o P4.28 / 10 MAT2.1: P0.7 / 11 o P4.29 / 10 MAT2.2: P0.8 / 11; MAT2.3: P0.9 / 11
3	CAP3.0: P0.23 / 11 CAP3.1: P0.24 / 11	MAT3.0: P0.10 / 11 MAT3.1: P0.11 / 11

Timer	PCONP: bit / reset	PCLKSEL: campo / reset
0	1 PCTIM0 / 1	PCLKSEL0[3:2] / 00
1	2 PCTIM1 / 1	PCLKSEL0[5:4] / 00
2	22 PCTIM2 / 0	PCLKSEL1[13:12] / 00
3	23 PCTIM3 / 0	PCLKSEL1[15:14] / 00

PCONP: 1 habilita reloj/alimentación del bloque; no inicia el conteo. En cada campo PCLKSEL: **00 = CCLK/4; 01 = CCLK; 10 = CCLK/2; 11 = CCLK/8**. Para contar tiempo o generar IRQ no es necesario sacar MAT a un pin.

LPC_TIMx: x = 0, 1, 2 o 3

Registro / reset	Bits	Función
TCR / R/W / 0	0 CE 1 CR	CE: 1 habilita TC y PC; 0 los detiene. CR: 1 resetea ambos en el próximo flanco de PCLK y los mantiene reseteados; debe volver a 0. 31:2 reservados.
CTCR / R/W / 0	1:0 CTM	00 = timer; 01 = cuenta flancos ascendentes; 10 = descendentes; 11 = ambos. Son eventos de conteo, no capturas a CR.
CTCR	3:2 CIS	00 = CAPx.0; 01 = CAPx.1; 10/11 reservados. 31:4 reservados. En contador, programar en 000 los tres bits de CCR de la entrada usada.
TC / R/W / 0	31:0	Contador de 32 bits. En modo timer incrementa cada PR+1 ciclos PCLK. Desborda de 0xFFFFFFFF a 0 sin IRQ automática.
PR / R/W / 0	31:0	Máximo de PC. En timer, divisor = PR+1. En contador usar PR=0 para contar cada evento seleccionado.
PC / R/W / 0	31:0	Prescaler: si PC=PR, el siguiente ciclo válido limpia PC e incrementa TC. En timer: PR=0 → un incremento de TC por PCLK.

Resolución en modo timer: $\Delta t_TC = (PR+1)/PCLK$. En modo contador, el manual limita la frecuencia de la señal CAP a PCLK/4. El otro canal CAP puede seguir capturando. La selección y puesta en cero de CCR las hace el software.

Timers: match, captura e interrupciones

Fuente: cap. 21, tablas 427, 430-433, pp. 504-510; tabla 532 (MAT0.3 interno).

Todos los campos de estos registros tienen reset 0. **IR, MR0..3, MCR, CCR y EMR:** R/W. **CR0 y CR1:** RO.

Registro	Bits	Operación
IR	0 / 1 / 2 / 3	Flags de interrupción MR0 / MR1 / MR2 / MR3, respectivamente.
IR	4 / 5	Flags de captura CR0 / CR1. Bits 31:6 reservados.
IR	0..5 (W1C)	Leer identifica pendientes; escribir 1 limpia la fuente y 0 no modifica. Limpiar un flag de match también limpia su solicitud DMA correspondiente.
MR0..MR3	31:0	Valor comparado con TC. Cada timer tiene cuatro registros de match, aunque no saque cuatro señales MAT a pines.

MCR: tres bits por match

Match	Interrumpir (I)	Reset TC (R)	Detener TC/PC (S)
MR0	0	1	2
MR1	3	4	5
MR2	6	7	8
MR3	9	10	11

Cada bit en 1 habilita su acción cuando TC=MRn; 0 la deshabilita. S también pone TCR[0]=0. Se pueden combinar acciones. MCR[31:12] reservado.

CCR y CR: captura del valor de TC

Entrada	Flanco ascendente	Flanco descendente	IRQ por captura
CAPx.0 → CR0	bit 0 CAP0RE	bit 1 CAP0FE	bit 2 CAP0I
CAPx.1 → CR1	bit 3 CAP1RE	bit 4 CAP1FE	bit 5 CAP1I

Flanco habilitado: copia TC a CR0/CR1[31:0]. I=1 solicita IRQ por esa captura; habilitar solo I no define un flanco. Ambos flancos pueden habilitarse. CCR[31:6] reservado. Leer CR no limpia IR.

EMR: estado y acción de las señales de match

Señal	Bit de estado	Campo de control
MATx.0	0 EM0	5:4 EMC0
MATx.1	1 EM1	7:6 EMC1
MATx.2	2 EM2	9:8 EMC2
MATx.3	3 EM3	11:10 EMC3

Cada EMC: **00 = no cambia; 01 = pone EM en 0; 10 = pone EM en 1; 11 = alterna EM** al match correspondiente. Fuera de 11:0 no se usan bits. El valor EM puede inicializarse por software.

EM2/EM3 y EMC2/EMC3 existen también en timers 0, 1 y 3. Solo el timer 2 dispone de MAT2.2 y MAT2.3 en pines. MAT0.3 es un ejemplo de señal interna utilizable para iniciar ADC. Para salida física seleccionar PINSEL.